

```
torch_device = "cuda"
vae.to(torch_device)
text_encoder.to(torch_device)
UNET.to(torch_device)
```

Next, we will specify the parameters that will be used to generate the images.

It's worth noting that `guidance_scale` is defined similarly to the guidance weight “*w*” in equation (2) of the Imagen paper. When `guidance_scale` is set to 1, no classifier-free guidance is performed. In this case, we have set `guidance_scale` to 7.5, as in the previous examples.

In contrast to previous examples, we will set `num_inference_steps` to 100 in order to obtain an image with even greater definition.

```
prompt = ["a photograph of an astronaut riding a horse"]
height = 512          # default height of Stable Diffusion
width = 512           # default width of Stable Diffusion
num_inference_steps = 100  # Number of denoising steps
guidance_scale = 7.5      # Scale for classifier-free guidance
generator = torch.manual_seed(0) # Seed generator to create the initial latent noise
```

```
batch_size = len(prompt)
```

To start, we obtain the text embeddings corresponding to the provided prompt. These embeddings will serve as a conditioning factor for the UNet model and assist in generating an image that closely resembles the given prompt.

```
text_input = tokenizer(prompt, padding="max_length", max_length=tokenizer.model_max_length, truncation=True, return_tensors="pt")
text_embeddings = text_encoder(text_input.input_ids.to(torch_device))[0]
```

We will obtain the unconditional text embeddings for guidance without using any classifier. These embeddings are simply the embeddings for the padding token, which represents empty text. They should have the same shape as the conditional `text_embeddings`, which includes `batch_size` and `seq_length`.

```
max_length = text_input.input_ids.shape[-1]
uncond_input = tokenizer(
    [""] * batch_size, padding="max_length", max_length=max_length,
    return_tensors="pt"
)
uncond_embeddings = text_encoder(uncond_input.input_ids.to(torch_device))[0]
```

To perform classifier-free guidance, we concatenate the conditioned input